

PROTOCOLE QUALITÉ

API Standard

Ce que « livré » veut dire pour une route API Leopardo RH

§01
Structure
&
Format

§02
Sécurité
&
Auth

§03
Validation
&
Erreurs

§04
Performance

§05
Tests
&
Docs

Périmètre :	Toutes les routes /api/v1/* et /api/ai/* du projet Leopardo RH
Obligation :	Aucune route ne peut être mergée sans satisfaire ce protocole à 100%
Bloquants :	Les critères marqués BLOQUANT empêchent le merge même si tout le reste passe
Amont :	Ce qui doit être fait AVANT d'écrire la première ligne de code
Livvable :	Ce qui doit exister AVANT d'ouvrir la Pull Request
Sections :	Structure · Sécurité · Validation · Performance · Tests · Checklist finale

5

Catégories

47

Critères

12

Bloquants

100%

Avant merge

PRÉAMBULE — CE QUE CE DOCUMENT IMPOSE

Ce protocole définit le standard minimum qu'une route API Leopard RH doit atteindre pour être considérée comme livrée. Il s'applique à toutes les routes — nouvelles ou modifiées. Il est non-négociable. Un développeur qui soumet une PR sans satisfaire ce protocole se verra retourner son travail sans revue de code.

Ce protocole couvre deux dimensions : (1) ce qui doit être pensé et fait **en amont**, avant d'écrire la première ligne de code, et (2) ce qui doit exister **comme livrable** avant d'ouvrir la Pull Request. L'ordre est important — les erreurs d'architecture se corrigent avant le code, pas après.

Niveau	Symbole	Signification	Conséquence si non satisfait
BLOQUANT	■	La route ne peut pas exister sans ça	Merge refusé automatiquement. Fix obligatoire.
REQUIS	■	Standard de qualité attendu	Commentaire de revue. Fix avant merge.
RECOMMANDE	■	Bonne pratique conseillée	Note dans la PR. Fix dans le sprint suivant.

§01

EN AMONT — AVANT D'ÉCRIRE UNE LIGNE

Ce qui doit être défini avant de coder

01

Définir le contrat avant le code

La règle d'or : le contrat de la route (ce qu'elle reçoit, ce qu'elle retourne, ce qu'elle fait) doit être écrit et validé AVANT de commencer l'implémentation. Un contrat mal pensé coûte 10x plus cher à corriger une fois le code écrit.

Pour chaque nouvelle route, le développeur doit répondre à ces questions AVANT de coder :

1. Quel est le besoin métier exact que cette route sert ?

Pas une question technique — une question business. 'Permettre au manager de voir qui est absent aujourd'hui.'

2. Qui peut appeler cette route et dans quelles conditions ?

Rôles autorisés. Plan minimum requis. Conditions spéciales (ex: seulement son propre tenant).

3. Qu'est-ce que la route reçoit en entrée ?

Paramètres de route, query params, body JSON. Types exacts. Contraintes (min, max, enum, nullable).

4. Qu'est-ce que la route retourne en cas de succès ?

Structure JSON exacte. Tous les champs. Types. Champs nullable vs obligatoires.

5. Quels sont les cas d'erreur possibles et leurs codes ?

Validation échoue. Ressource non trouvée. Permission refusée. Erreur serveur. Chaque cas = code HTTP + error_code.

6. Est-ce que cette route peut avoir des effets de bord ?

Envoie-t-elle un email ? Notifie-t-elle ? Crée-t-elle un audit log ? Modifie-t-elle d'autres tables ?

7. Comment cette route se comporte-t-elle sous charge ?

Est-elle idempotente ? Peut-on la caller 2 fois de suite sans problème ? Quel est le worst-case en DB ?

8. Quels tests Pest valideront que c'est correct ?

Lister les cas de test AVANT d'écrire le controller. TDD ou au moins contract-first.

Décider du design RESTful AVANT de coder

Décision	Règle Leopardo RH	Exemple correct	Exemple incorrect
Nommage des routes	Noms de ressources au pluriel, kebab-case	GET /api/v1/attendance-logs	GET /api/v1/getAttendanceLogs
Verbes HTTP	GET=lecture · POST=création · PUT=remplacement · PATCH=modification partielle · DELETE=suppression	PATCH /api/v1/employees/{id}	POST /api/v1/updateEmployee
Versioning	Toujours /api/v1/ · Jamais changer une route existante sans versionner	POST /api/v2/employees/invite si breaking change	Modifier /api/v1/employees/invite directement
Paramètres de filtre	Query params pour les filtres, pas dans le body d'un GET	GET /attendance?date=2026-04-01&status;=late	POST /attendance/filter avec body
Relations imbriquées	Maximum 2 niveaux	GET /companies/{id}/employees	GET /companies/{id}/departments/{id}/employees/{id}/leaves
Format des dates	ISO 8601 UTC en sortie TOUJOURS	2026-04-24T08:00:00Z	24/04/2026 08:00 ou timestamp Unix
Pagination	Toujours paginer les listes. Default 20 items. Max 100.	GET /employees?page=2&per_page=20	Retourner tous les employees sans limite

§02

STRUCTURE & FORMAT — LE CONTRAT JSON

Enveloppe · Pagination · Champs · Conventions

02

L'enveloppe de réponse — format unique et cohérent

Toutes les réponses API Leopard RH utilisent la même enveloppe JSON. Sans exception. Jamais retourner un objet nu ou un array nu à la racine.

Enveloppe JSON — format unique obligatoire

```
// Réponse de SUCCÈS — structure obligatoire
{
  "data": { ... },           // L'objet ou l'array retourné
  "message": "string",       // Message humain optionnel (pour les actions)
  "meta": {                  // Présent SEULEMENT si liste paginée
    "current_page": 1,
    "per_page": 20,
    "total": 145,
    "last_page": 8,
    "from": 1,
    "to": 20
  }
}

// Réponse d'ERREUR — structure obligatoire
{
  "error_code": "ALREADY_CHECKED_IN", // Snake UPPER_CASE — jamais HTTP status text
  "message": "Vous avez déjà pointé votre arrivée aujourd'hui.", // Traduit via __('errors.CODE')
  "errors": {                  // Présent SEULEMENT si erreur de validation (422)
    "email": ["L'email est invalide."],
    "start_date": ["La date de début est requise."]
  }
}

// CE QUI EST INTERDIT en réponse :
// - Retourner true / false / null seul à la racine
// - Retourner un array [] à la racine
// - Retourner un objet sans clé 'data'
// - Inclure des stack traces en production
// - Inclure des données sensibles (password_hash, iban, national_id)
```

Codes HTTP — table de référence

Code	Nom	Quand l'utiliser	Leopardo RH exemple
200	OK	Requête réussie avec données retournées	GET /attendance/today → 200 + data

Code	Nom	Quand l'utiliser	Leopardo RH exemple
201	Created	Ressource créée avec succès	POST /employees/invite → 201 + data
204	No Content	Action réussie sans données à retourner	DELETE /sessions/{id} → 204 (pas de body)
400	Bad Request	Requête malformée (JSON invalide, params manquants)	POST sans Content-Type JSON
401	Unauthorized	Token absent, invalide ou expiré	Requête sans header Authorization
403	Forbidden	Authentifié mais pas autorisé	Employé qui appelle une route manager
404	Not Found	Ressource introuvable dans le tenant	GET /employees/999 (autre tenant)
409	Conflict	Conflit avec l'état actuel	Double check-in → 409 ALREADY_CHECKED_IN
422	Unprocessable Entity	Validation échouée (données invalides)	Date de fin < date de début → 422
429	Too Many Requests	Rate limit dépassé	Trop de tentatives de login
500	Internal Server Error	Erreur non gérée du serveur	Exception non catchée → log + 500

Conventions de nommage des champs JSON

Conventions nommage champs JSON

// Conventions champs JSON – à respecter dans tous les serializers

// ■ CORRECT – snake_case pour tous les champs

```
{
  "employee_id": 42,
  "check_in": "2026-04-24T07:58:23Z",    // ISO 8601 UTC – TOUJOURS
  "hours_worked": 8.5,                  // Décimal, jamais int
  "is_late": true,                       // Booléen – préfixe is_ ou has_
  "created_at": "2026-04-24T08:00:00Z",  // Timestamp UTC
  "company_id": "uuid-...",             // UUID pour les entités publiques
}
```

// ■ INTERDIT

```
{ "employeeId": 42 }           // camelCase – NON
{ "CheckIn": "..." }         // PascalCase – NON
{ "check_in": 1745481503 }     // Timestamp Unix – NON (ISO 8601 uniquement)
{ "hours": "8h30" }           // String formatée – NON (nombre décimal)
{ "password": "hash..." }    // Données sensibles – JAMAIS dans la réponse
```

§03

SÉCURITÉ & AUTHENTIFICATION

RBAC · Tenant isolation · Rate limiting · Headers

03

La sécurité n'est pas une feature — c'est un prérequis. Une route non sécurisée n'est pas une route livrée. Les 4 vérifications de sécurité (auth, permission, tenant, rate limit) doivent être appliquées dans CET ORDRE pour chaque route protégée.

Les 4 couches de sécurité — ordre obligatoire

Les 4 couches de sécurité — ordre et implémentation

// Chaque route protégée suit cette pile middleware dans l'ordre :

```
Route::middleware([
    'auth:sanctum',           // 1. TOKEN VALIDE ? — 401 si absent ou expiré
    'set.locale',             // 2. LOCALE — définit la langue pour les messages
    'tenant.inject',          // 3. TENANT CONTEXT — charge la company dans le scope
    'throttle:api',           // 4. RATE LIMIT — 60 req/min par défaut (configurable)
])->group(function() {
    Route::get('/attendance/today', [AttendanceController::class, 'today']);
});
```

// Dans le Controller — vérification de permission :

```
public function today(Request $request): JsonResponse {
    // 5. PERMISSION — le rôle peut-il accéder à cette action ?
    $this->authorize('viewAttendance', Employee::class);
    // 6. DONNÉES — uniquement dans le tenant courant (Global Scope automatique)
    $data = AttendanceLog::today()->get(); // Global Scope filtre company_id auto
    return response()->json(['data' => AttendanceResource::collection($data)]);
}
```

RBAC — matrice de vérification

Règle	Implémentation	Vérification	BLOQUANT ?
Chaque route a au moins 1 rôle autorisé défini	Gate ou Policy Laravel + middleware authorize()	Test Pest : appel avec rôle non autorisé → 403	Oui
Un employé ne voit jamais les données d'un autre employee	Policy + Global Scope company_id	Test : employee_A essaie d'accéder aux données d'employee_B → 403 ou 404	Oui
Les données tenant sont strictement isolées	Global Scope automatique sur tous les modèles tenant	Test : token company_A + route → 0 donnée de company_B	Oui
Les routes manager ne sont pas accessibles aux employés	Policy ou middleware role check	Test : employé appelle route manager → 403	Oui

Règle	Implémentation	Vérification	BLOQUANT ?
Les routes super_admin ne sont pas accessibles aux tenants	Middleware IsSuperAdmin séparé	Test : token tenant sur route /admin/* → 403	Oui
Pas de permission ROLE hardcodée dans le controller	Utiliser les Gates/Politiques, jamais if (\$user->role === 'manager')	Code review — grep pour hardcoded role checks	Requis

Headers de sécurité HTTP — vérification

Headers de sécurité HTTP

```
// Headers que l'API Leopardo RH doit retourner sur CHAQUE réponse
// Configurer dans app/Http/Middleware/SecurityHeaders.php

Content-Type: application/json; charset=UTF-8 // TOUJOURS JSON
X-Content-Type-Options: nosniff // Empêche le MIME sniffing
X-Frame-Options: DENY // Empêche le clickjacking
Referrer-Policy: strict-origin-when-cross-origin

// Sur les routes qui modifient des données :
// Vérifier que le Content-Type de la requête est application/json
// Sinon → 415 Unsupported Media Type

// CE QUI NE DOIT PAS APPARAÎTRE dans les headers de réponse :
// X-Powered-By: PHP/8.4 ← expose la version PHP
// Server: nginx/1.25.3 ← expose la version Nginx
// Désactiver dans nginx.conf : server_tokens off;
// Désactiver dans PHP : expose_php = Off
```

Rate limiting — configuration par type de route

Type de route	Limite	Clé de rate limit	Réponse si dépassé
Routes publiques (sans auth)	20 req/min	IP uniquement	429 + header Retry-After
Routes auth standard	60 req/min	IP + user_id	429 + header Retry-After
Routes de login/auth	5 tentatives/5min	IP + email (ciblé)	429 + locked_until si > 10 tentatives
Routes AI chat	Selon plan (1 0-infini/jour)	company_id + date	429 + quota info + lien upgrade
Routes upload/génération PDF	10 req/min	user_id	429 + message délai
Routes webhook/public	30 req/min	IP	429

S04

VALIDATION & GESTION DES ERREURS

FormRequest · Messages traduits · Cas d'erreur exhaustifs

04

FormRequest — la validation passe toujours par là

Aucune validation ne se fait dans le Controller. Toujours utiliser un FormRequest Laravel dédié. Le FormRequest valide les types, les contraintes, les règles métier simples et retourne les erreurs traduites automatiquement.

CreateAbsenceRequest.php — FormRequest complet

```
// app/Http/Requests/Api/V1/CreateAbsenceRequest.php
class CreateAbsenceRequest extends FormRequest {
    public function authorize(): bool {
        // Vérification de permission ICI (pas dans le controller)
        return $this->user()->can('create', Absence::class);
    }

    public function rules(): array {
        return [
            'employee_id' => ['required', 'integer', Rule::exists('employees', 'id')],
            // exists() respecte le Global Scope — impossible de cibler un autre tenant
            'absence_type' => ['required', Rule::in(['annual_leave', 'sick_leave', 'unpaid', 'other'])],
            'start_date'   => ['required', 'date', 'after_or_equal:today'],
            'end_date'     => ['required', 'date', 'after_or_equal:start_date'],
            'reason'       => ['nullable', 'string', 'max:500'],
        ];
    }

    public function messages(): array {
        // Tous les messages passent par les fichiers lang/
        return [
            'employee_id.exists' => __('validation.employee_not_found'),
            'start_date.after_or_equal' => __('validation.date_in_past'),
        ];
    }

    protected function failedValidation(Validator $validator): void {
        // Forcer la réponse JSON (Laravel retourne HTML par défaut si pas Accept: application/json)
        throw new HttpResponseException(response()->json([
            'error_code' => 'VALIDATION_ERROR',
            'message'    => __('errors.VALIDATION_ERROR'),
            'errors'     => $validator->errors(),
        ], 422));
    }
}
```

Gestion des erreurs — handler global


```

Handler d'exceptions global — JSON toujours
// bootstrap/app.php — handler d'exceptions global
->withExceptions(function (Exceptions $exceptions) {
    // Toutes les exceptions non gérées → JSON structuré (jamais HTML)
    $exceptions->render(function (Throwable $e, Request $request) {
        if ($request->expectsJson() || str_starts_with($request->path(), 'api/')) {
            // ModelNotFoundException → 404 propre
            if ($e instanceof ModelNotFoundException) {
                return response()->json(['error_code'=>'NOT_FOUND',
                    'message'=>__('errors.NOT_FOUND')], 404);
            }
            // AuthorizationException → 403 propre
            if ($e instanceof AuthorizationException) {
                return response()->json(['error_code'=>'FORBIDDEN',
                    'message'=>__('errors.FORBIDDEN')], 403);
            }
            // Toute autre exception → 500 sans stack trace en production
            Log::error($e->getMessage(), ['trace' => $e->getTraceAsString()]);
            return response()->json(['error_code'=>'SERVER_ERROR',
                'message'=>__('errors.SERVER_ERROR')], 500);
        }
    });
});
})

```

Exhaustivité des cas d'erreur — obligation

Pour chaque route, TOUS les cas d'erreur doivent être identifiés et documentés dans le contrat de la route. Un cas d'erreur non documenté est un cas d'erreur non testé.

```

Exhaustivité des cas d'erreur — contrat check-in
// Exemple : POST /api/v1/attendance/check-in
// Cas de succès :
// 201 → Check-in créé avec succès

// Cas d'erreur — TOUS doivent être testés :
// 401 → Token absent ou invalide (middleware)
// 403 → Rôle manager essaie de pointer (policy)
// 403 → Compte suspendu ou archivé (middleware)
// 409 → ALREADY_CHECKED_IN — déjà pointé aujourd'hui
// 422 → GPS invalide si obligatoire sur ce plan
// 429 → Rate limit dépassé
// 500 → Erreur inattendue (loguée, pas de détail en prod)

// Pour chaque cas d'erreur :
// 1. Code HTTP correct
// 2. error_code en UPPER_SNAKE_CASE
// 3. Message traduit via __('errors.CODE')
// 4. Test Pest qui vérifie exactement ce comportement

```

§05

PERFORMANCE — SEUILS & BONNES PRATIQUES

Temps de réponse · N+1 · Index · Cache

05

Seuils de performance acceptables

Type de route	Seuil p95 acceptable	Seuil p99 acceptable	Si dépassé
GET simple (1 objet)	< 100ms	< 200ms	Ajouter index · Vérifier N+1
GET liste paginée (20 items)	< 200ms	< 400ms	Ajouter index · Eager loading
GET avec agrégation (Daily Summary)	< 500ms	< 1000ms	Vue matérialisée · Cache
POST création simple	< 150ms	< 300ms	Vérifier les queries dans la transaction
POST avec email/notification	< 200ms	< 500ms	Queuer l'email — ne pas attendre l'envoi
GET génération PDF	< 2000ms	< 4000ms	Cache le PDF · Job async si > 2s
Routes AI (LLM call)	< 3000ms	< 8000ms	Streaming si > 3s · Timeout 30s
Voice STT+TTS pipeline	< 3000ms	< 6000ms	Streaming audio · Paralléliser STT/processing

Le problème N+1 — détection et correction obligatoire

Une route avec un problème N+1 est une route non livrée. Avant chaque PR, activer les query logs Laravel et vérifier que le nombre de queries est constant (ne croît pas avec le nombre d'items retournés).

Détection et correction N+1 — obligatoire avant PR

```
// ■ N+1 — 1 query pour la liste + 1 query par employé = problème critique
$employees = Employee::all();
foreach ($employees as $emp) {
    $emp->attendance_today; // 1 query par employé → 50 employés = 51 queries !
}

// ■ CORRECT — eager loading — 2 queries total quelle que soit la taille
$employees = Employee::with(['attendanceToday', 'department'])->active()->get();

// ■ CORRECT — agréger en SQL
$employees = Employee::select('employees.*')
    ->withCount(['absences as absences_this_month' =>
        fn($q) => $q->whereMonth('start_date', now()->month)
    ])
    ->active()->get();

// VÉRIFICATION : activer le log dans le test ou le tinker
DB::enableQueryLog();
// ... appeler la route ...
$queryes = DB::getQueryLog();
// Vérifier que count($queryes) est <= 5, quelle que soit la taille des données
```

Index PostgreSQL — règles

Index PostgreSQL — obligatoires par table

```
// Règle : chaque colonne utilisée dans WHERE, ORDER BY ou JOIN fréquent doit avoir un index

// Index obligatoires sur toutes les tables tenant :
CREATE INDEX ON attendance_logs (company_id, date DESC); // Toujours filtré par tenant + date
CREATE INDEX ON attendance_logs (employee_id, date DESC); // Profil employé
CREATE INDEX ON employees (company_id, status); // Liste employés actifs
CREATE INDEX ON absences (company_id, start_date, end_date); // Conflits de dates

// Vérifier avec EXPLAIN ANALYZE avant de livrer une query complexe :
EXPLAIN ANALYZE
SELECT * FROM attendance_logs
WHERE company_id = 'uuid' AND date = '2026-04-24'
ORDER BY created_at DESC;
-- Doit montrer 'Index Scan' — jamais 'Seq Scan' sur une grande table

// Index partiels — pour les cas fréquents :
CREATE INDEX ON attendance_logs (company_id, date) WHERE check_out IS NULL;
-- Accélère 'Qui n'a pas encore pointé sa sortie aujourd'hui'
```

§06

TESTS & DOCUMENTATION

Tests Pest obligatoires · Contrat API · OpenAPI

06

Tests Pest — ce qui doit exister avant le merge

La règle : pour chaque route livrée, il doit exister un fichier de test Pest qui couvre au minimum les cas listés ci-dessous. Un test qui échoue = une PR qui ne merge pas.

```
// tests/Feature/Api/V1/AttendanceTest.php – structure type
```

[illegible]

Coverage minimum par type de test

Type de test	Coverage minimum	Outil	Quand exécuter
Tests unitaires (Services, Actions)	80% des méthodes	Pest Unit	Sur chaque commit (pre-commit hook)
Tests feature (routes API)	100% des cas d'erreur listés dans le contrat	Pest Feature	Sur chaque PR avant review
Tests d'intégration (DB + Redis)	Chaque query critique testée avec données réelles	Pest Feature + RefreshD atabase	Sur chaque PR
Tests de régression (routes modifiées)	Les anciens tests continuent de passer	Pest (suite complète)	Avant chaque merge sur main
Tests de performance (seuils)	p95 dans les seuils définis en §05	k6 ou Pest timing assert	Avant chaque release (pas chaque PR)

Documentation de la route — obligatoire

Chaque route doit être documentée dans le fichier `docs/dossierdeConception/01_API_CONTRATS_COMPLETS/02_API_CONTRATS_COMPLET.md`. La documentation doit contenir au minimum :

Format documentation route — Markdown obligatoire

POST /api/v1/attendance/check-in

****Objectif**** : Enregistrer l'arrivée d'un employé pour la journée courante.

****Auth**** : Bearer token Sanctum | ****Rôles**** : employee

****Rate limit**** : 60/min

Body (optionnel)

```json

```
{ "gps_lat": 36.752887, "gps_lng": 3.042048 }
```

```

Réponses

Code	error_code	Condition
------	------------	-----------

201	—	Check-in créé, premier de la journée
-----	---	--------------------------------------

401	UNAUTHENTICATED	Token absent ou invalide
-----	-----------------	--------------------------

403	FORBIDDEN	Rôle non autorisé (manager)
-----	-----------	-----------------------------

403	ACCOUNT_SUSPENDED	Compte suspendu
-----	-------------------	-----------------

409	ALREADY_CHECKED_IN	Déjà pointé aujourd'hui
-----	--------------------	-------------------------

429	RATE_LIMITED	Trop de requêtes
-----	--------------	------------------

Réponse 201

```json

```
{
```

```
 "data": {
```

```
 "id": 42, "employee_id": 7, "date": "2026-04-24",
```

```
 "check_in": "2026-04-24T07:58:23Z", "check_out": null,
```

```
 "status": "ontime", "late_minutes": 0, "hours_worked": null
```

```
 }
```

```
}
```

```

§07

CHECKLIST AVANT PULL REQUEST

Les 47 critères · Bloquants en rouge · Cocher avant de soumettre

07

Cette checklist est le dernier filtre avant d'ouvrir une Pull Request. Le développeur la remplit lui-même. Le reviewer vérifie les cases critiques en premier. Si un item BLOQUANT est non coché → la PR est refusée sans revue.

Catégorie 1 — Contrat & Design (EN AMONT)

- ☐ **Le contrat de la route est écrit AVANT le code (entrées, sorties, erreurs)** BLOQUANT
Fichier CONTRAT-`[ROUTE]`.md ou section dans le doc API
- ☐ **La route suit les conventions REST Leopard RH (verbe, URL, nommage)** BLOQUANT
Vérifier §02 de ce document
- ☐ **Toutes les erreurs possibles sont identifiées et ont un `error_code` défini** BLOQUANT
Minimum 3 cas d'erreur par route
- ☐ **Le format de réponse utilise l'enveloppe standard `{data, message, meta}`** BLOQUANT
Jamais de réponse nue
- ☐ **Les dates retournées sont en ISO 8601 UTC** BLOQUANT
Jamais de timestamp Linux ni de date formatée
- ☐ **Les champs JSON sont en snake_case**
Vérifier tous les serializers
- ☐ **Pas de données sensibles dans la réponse (password, iban, national_id)** BLOQUANT
Grep dans les Resources Laravel

Catégorie 2 — Sécurité & Auth

- ☐ **La route a le middleware `auth:sanctum`** BLOQUANT
Vérifier dans `routes/api.php`
- ☐ **La route a une Policy ou Gate définie et testée** BLOQUANT
Pas de vérification inline dans le controller
- ☐ **L'isolation tenant est garantie (Global Scope sur tous les modèles utilisés)** BLOQUANT
Vérifier chaque Model utilisé
- ☐ **Rate limiting configuré et approprié au type de route** BLOQUANT
Voir tableau §03
- ☐ **Aucune donnée d'un autre tenant n'est accessible même avec un ID direct** BLOQUANT
Test Pest cross-tenant obligatoire
- ☐ **Les headers de sécurité sont présents dans toutes les réponses**
Middleware `SecurityHeaders` global
- ☐ **Pas de role check hardcodé dans le controller (if `$user->role === 'manager'`)**
Utiliser les Gates

Catégorie 3 — Validation & Erreurs

☐	Un FormRequest dédié est utilisé pour valider les inputs Jamais de validation dans le controller	BLOQUANT
☐	Tous les champs requis sont validés avec les bons types required_string_integer_date_etc	BLOQUANT
☐	Les messages d'erreur de validation sont traduits (lang/fr/validation.php) (*validation xxx*)	
☐	Le handler global retourne du JSON sur les exceptions non gérées Vérifier bootstrap/ann.php	BLOQUANT
☐	Chaque cas d'erreur retourne le bon code HTTP ET le bon error_code Voir table \$QA	BLOQUANT
☐	Les erreurs de validation retournent un objet 'errors' structuré 422 avec errors par champ	BLOQUANT
☐	Pas de stack trace dans les réponses de production APP_DEBUG=false en staging/prod	BLOQUANT

Catégorie 4 — Performance

☐	Pas de problème N+1 (vérifier avec DB::getQueryLog()) Max 5 queries pour toute liste	BLOQUANT
☐	Les index PostgreSQL nécessaires existent sur les colonnes filtrées EXPI AIN ANALYZE sur les queries complexes	BLOQUANT
☐	Les listes sont paginées (max 100 items par page) default 20 max 100	BLOQUANT
☐	Les emails et notifications sont dans des jobs Queue (pas synchrones) Mail::queue() pas Mail::send()	BLOQUANT
☐	Le temps de réponse p95 est dans les seuils définis en \$Q5 Tester avec k6 ou mesurer en dev	
☐	Les PDFs et exports lourds sont générés de manière asynchrone Job Queue + polling ou webhook	

Catégorie 5 — Tests

☐	Test Pest : cas de succès nominal (bonne entrée → bon résultat) Minimum 1 test succès	BLOQUANT
☐	Test Pest : 401 sans token Obligatoire sur toutes les routes protégées	BLOQUANT
☐	Test Pest : 403 rôle non autorisé Au moins 1 rôle non autorisé testé	BLOQUANT
☐	Test Pest : isolation tenant (données cross-tenant inaccessibles) Test avec 2 companies différentes	BLOQUANT
☐	Test Pest : tous les cas d'erreur métier (409, 422 spécifiques) Chaque error_code doit avoir 1 test	BLOQUANT
☐	Test Pest : horodatage serveur (si applicable) Vérifier que le timestamp client est ignoré	BLOQUANT
☐	Tous les tests existants continuent de passer (pas de régression) php artisan test --no-coverage passe à 0 erreur	BLOQUANT
☐	Coverage > 80% sur le code de la route (Controller + Service + Action) php artisan test --coverage	

Catégorie 6 — Documentation

☐	La route est documentée dans 02_API_CONTRATS_COMPLET.md Format défini en 806	BLOQUANT
☐	Les error_codes sont ajoutés dans lang/fr/errors.php ET traduits dans AR/TR/EN Chaque error_code doit avoir 4 traductions	BLOQUANT
☐	La migration (si nouvelle table/colonne) est backward-compatible Pas de DROP : Pas de NOT NULL sans DEFAULT	BLOQUANT
☐	Le CHANGELOG.md est mis à jour avec la nouvelle feature Format SemVer — ajout ou modification	
☐	PILOTAGE.md est mis à jour si la feature était dans un sprint planifié Marquer la tâche comme livrée	

VALIDATION PULL REQUEST

Je certifie que tous les critères BLOQUANTS sont satisfaits et que cette Pull Request respecte le Protocole Qualité API Leopardo RH v1.0.

Développeur

Route(s) livrée(s)

Date

Reviewer

Protocole Qualité API Leopardo RH v1.0 — Avril 2026. Ce document évolue avec le projet. Toute exception au protocole doit être documentée et approuvée par le lead technique avant le merge. Zéro exception non documentée.